

CHAIRMAN CODE SNIPPET

1. Login Authentication & Security(chair/vits/rep)

```
# Hashing & Validation Helpers
def hash_password(raw: str) -> str:
    return make_password(raw)

def verify_password(raw: str, stored: str) -> bool:
    if not stored:
        return False
    if stored.startswith('pbkdf2_'):
        return check_password(raw, stored)
    return hashlib.sha256(raw.encode()).hexdigest() == stored

# 5x Failed Attempt Security
def is_ip_locked(ip):
    return (cache.get(f'login_attempts_{ip}') or 0) >= 5

def track_login_failure(ip):
    key = f'login_attempts_{ip}'
    current = cache.get(key, 0)
    cache.set(key, current + 1, timeout=900) # Locks for 15 minutes

def clear_login_failures(ip):
    cache.delete(f'login_attempts_{ip}')

# Login Endpoint (Snippet)
def login_view(request):
    # ...
    ip = get_client_ip(request)
```

```

# ...
ip = get_client_ip(request)
if is_ip_locked(ip):
    messages.error(request, 'Too many failed attempts. Try again after 15
minutes.')
    return render(request, 'admin_panel/login.html', {'form': form})

if form.is_valid():
    email = form.cleaned_data['email']
    password = form.cleaned_data['password']
    try:
        admin = Admin.objects.get(email=email, is_active=True)
        if not verify_password(password, admin.password_hash):
            raise Admin.DoesNotExist
        # Automatic PBKDF2 Hash Migration
        if not admin.password_hash.startswith('pbkdf2_'):
            admin.password_hash = make_password(password)
            admin.save(update_fields=['password_hash'])
        clear_login_failures(ip)
        # ... successful login session assignment ...
    except Admin.DoesNotExist:
        messages.error(request, 'Invalid email or password.')
        track_login_failure(ip)

```

2. Event management (chair/vits/) & rep except from creating event

```

# Creating an Event & Setting Expected Sections
def event_create_view(request):
    # ...
    event = form.save(commit=False)
    event.id = uuid.uuid4()
    event.created_by = get_current_admin(request)
    event.save()

    # Assigning expected sections to the event
    choice = form.cleaned_data.get('expected_year_section', '')
    if choice.startswith('year_'):
        year_level = int(choice.replace('year_', ''))
        event.expected_sections.set(Section.objects.filter(year_level=year_level))
    elif choice.startswith('section_'):
        section_id = choice.replace('section_', '')
        event.expected_sections.set(Section.objects.filter(id=section_id))
    else:
        event.expected_sections.clear() # Open to all

# Start and End Event
def start_event_view(request, pk):
    event = get_object_or_404(Event, id=pk)
    event.status = 'active'

```

```

    event = get_object_or_404(Event, id=pk)
    event.status = 'active'
    if not event.start_time:
        event.start_time = dj_timezone.now().time()
    event.save(update_fields=['status', 'start_time'])

def end_event_view(request, pk):
    event = get_object_or_404(Event, id=pk)
    event.status = 'ended'
    event.save(update_fields=['status'])

# Finalize (Lock) Event
def finalize_event_view(request, event_id):
    event = get_object_or_404(Event, id=event_id)
    absent_logs = AttendanceLog.objects.filter(event=event, status='absent')
    count = absent_logs.count()
    messages.success(request, f'Event "{event.name}" finalized – {count} student(s)
marked as Absent.')
    return redirect('admin_panel:attendance', event_id=event_id)

# Excel Export & Summary of Attendance
def export_attendance_view(request, event_id):
    event = get_object_or_404(Event, id=event_id)
    logs = AttendanceLog.objects.filter(event=event).select_related('student',
'scanned_by').order_by('scanned_at')

    # ... Excel styling setup ...

```

```

    # ... Excel styling setup ...
    # Appending summary rows
    summary_row = ws.max_row + 2
    ws.cell(row=summary_row, column=1, value='Summary').font = Font(bold=True)
    ws.cell(row=summary_row, column=2, value=f'Present:
{logs.filter(status="present").count()}')
    ws.cell(row=summary_row, column=3, value=f'Late:
{logs.filter(status="late").count()}')
    ws.cell(row=summary_row, column=4, value=f'Absent:
{logs.filter(status="absent").count()}')
    ws.cell(row=summary_row, column=5, value=f'Total: {logs.count()}')

```

3. Officers & Representatives Adding, Deleting, and One-Time Random Password Emailing

```

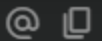
# Adding an Admin / VITS Rep & Sending One Time Password
def admins_manage_view(request):
    if action == 'add':
        # ... validation ...
        generated_password = secrets.token_urlsafe(12) # One time random password
        new_admin = Admin.objects.create(
            id=uuid.uuid4(),
            name=name,
            email=email,
            password_hash=hash_password(generated_password),
            role=role,
            is_active=True,
            force_password_change=True, # Forces change on login
        )
        # Email sending logic
        subject = f'CODE-IT Admin Account – Your Login Credentials'
        body = (f'Email: {email}\nTemporary Password: {generated_password}\nRole:
{role.title()}')
        send_mail(subject, body, settings.DEFAULT_FROM_EMAIL, [email],
fail_silently=True)

    # Deleting an Admin
    elif action == 'delete':
        admin = Admin.objects.get(id=admin_id)
        if admin.role != Admin.Role.CHAIRPERSON:
            admin.delete()

```

4. Section Management Adding and Deleting Sections

python



```
# Bulk Add Sections
def sections_manage_view(request):
    if request.method == 'POST':
        year_level = int(request.POST.get('year_level', '1'))
        count = int(request.POST.get('count', '1'))

        for _ in range(count):
            existing = Section.objects.filter(year_level=year_level)
            max_idx = 0
            for s in existing:
                m = _re.match(r'BSIT\s+\d+--(\d+)', s.name, _re.IGNORECASE)
                if m and int(m.group(1)) > max_idx:
                    max_idx = int(m.group(1))

            section_name = f'BSIT {year_level}-{max_idx + 1}'
            if not Section.objects.filter(name__iexact=section_name).exists():
                Section.objects.create(id=uuid.uuid4(), name=section_name,
year_level=year_level)

# Delete Section
def delete_section_view(request, pk):
    section = Section.objects.get(id=pk)
    section.delete()
```

5. Students Management Approving, Sending QR via Email (SMS Equivalent), Revoking, and Filtering (chair/vits)

```

# Filtering & Viewing Students by Status and Section
def students_view(request):
    status_filter = request.GET.get('status', 'all')
    section_filter = request.GET.get('section', 'all')

    students =
Student.objects.filter(section__in=assigned_sections).select_related('section').order
_by('-created_at')

    if status_filter in ('pending', 'active', 'rejected'):
        students = students.filter(status=status_filter)
    if section_filter != 'all':
        students = students.filter(section_id=section_filter)

# Approving Student & Emailing QR Token (Acts as SMS/Email delivery)
def student_approve_view(request, pk):
    student = get_object_or_404(Student, id=pk)
    student.status = 'active'
    student.save()

    # Generate QR Token
    token_value = secrets.token_urlsafe(32)
    qr_token = QRToken.objects.create(id=uuid.uuid4(), student=student,
token=token_value, qr_path='')
    # ... qrcode generation logic saving to BytesIO ...

```

```

# Send Email with attached QR code
email = EmailMessage(
    subject=f'Your CODE-IT QR Code - {student.name}',
    body='Your account has been approved! Show it at events.',
    from_email=settings.EMAIL_HOST_USER,
    to=[student.email]
)
email.attach('qr_code.png', buf.getvalue(), 'image/png')
email.send(fail_silently=False)

# Revoke Approval / QR Code
def revoke_qr_view(request, pk):
    student = get_object_or_404(Student, id=pk)
    QRToken.objects.filter(student=student).delete() # Delete old token
    new_token = secrets.token_urlsafe(32)
    qr_token = QRToken.objects.create(id=uuid.uuid4(), student=student,
token=new_token, qr_path='')
    # ... Regenerate and send new QR code via email ...

```

6. Dashboard Analytics Section Overview, Event Filter, Graphs, and Counting Students per Section(chair/vits)

```
# Dashboard Analytics & Counting
def dashboard_view(request):
    total_students = Student.objects.filter(status='active',
section__in=assigned_sections).count()
    pending_students = Student.objects.filter(status='pending',
section__in=assigned_sections).count()

    # Event filter for section graphs
    selected_event_id = request.GET.get('event')
    selected_event = Event.objects.get(id=selected_event_id) if selected_event_id
else None

    # Section Overview (Counting students, present, late, absent per section)
    section_cards = []
    for section in assigned_sections.order_by('year_level', 'name'):
        students = Student.objects.filter(section=section)

        if selected_event:
            section_logs = AttendanceLog.objects.filter(student__section=section,
event=selected_event)
        else:
            section_logs = AttendanceLog.objects.none()

        present = section_logs.filter(status='present').count()
        late = section_logs.filter(status='late').count()
        absent = section_logs.filter(status='absent').count()
```

```
absent = section_logs.filter(status='absent').count()

section_cards.append({
    'name': section.name,
    'total_students': students.count(),
    'active_students': students.filter(status='active').count(),
    'pending_students': students.filter(status='pending').count(),
    'present': present,
    'late': late,
    'absent': absent,
    # Calculating heights for the UI Graphs based on attendance count
    'present_height': min(260, present * 7),
    'late_height': min(260, late * 7),
    'absent_height': min(260, absent * 7),
})
```

7. ACTIVITY LOGS


```
# --- 1. CORE TRACKING FUNCTIONS ---
```

```
def get_current_admin(request):
    admin_id = request.session.get('admin_id')
    if admin_id:
        try:
            return Admin.objects.get(id=admin_id)
        except Admin.DoesNotExist:
            pass
    return None

def log_activity(request, action, target=None, description=None):
    admin = get_current_admin(request)
    try:
        ActivityLog.objects.create(
            id=uuid.uuid4(),
            admin=admin,          # Tracks WHO did it
            action=action,        # Action Type (e.g., 'ADD_SECTION', 'LOGS_CLEARED')
            target=target,        # What was affected
            description=description, # Specific details
            # created_at (Date) is automatically saved by Django via
            auto_now_add=True in models.py
        )
    except Exception:
        pass
```

```
# --- 2. EXAMPLE OF TRACKING BEING USED (e.g., Deleting an Event) ---
```

```
def event_delete_view(request, pk):
    event = get_object_or_404(Event, id=pk)
    name = event.name

    # Invokes the tracking function right before deletion
    log_activity(request, 'EVENT_DELETED', target=name, description=f'Deleted event "{name}"')
    return HttpResponseRedirect(reverse('event_list'))
```

```

# Invokes the tracking function right before deletion
log_activity(request, 'EVENT_DELETED', target=name, description=f'Deleted event "{name}"')

event.delete()

# --- 3. FILTERING & CLEARING LOGS VIEW ---

def activity_logs_view(request):
    # Verify Permissions
    if request.session.get('admin_role') not in ('chairperson', 'vits'):
        messages.error(request, 'You do not have permission to view activity logs.')
        return redirect('admin_panel:dashboard')

    # Fetch all logs, ordering by newest first
    logs = Activitylog.objects.select_related('admin').order_by('-created_at')

    # Get filter values from URL parameters
    action_filter = request.GET.get('action', 'all')
    admin_filter = request.GET.get('admin_id', 'all')
    date_from = request.GET.get('date_from', '')
    date_to = request.GET.get('date_to', '')

    # Filter by Action Type
    if action_filter != 'all':
        logs = logs.filter(action=action_filter)

    # Filter by Admin/Representative
    if admin_filter != 'all':
        logs = logs.filter(admin_id=admin_filter)

    # Filter by Date Range
    if date_from:
        try:
            logs = logs.filter(created_at__date__gte=date_from)

```

```

# Filter by Date Range
if date_from:
    try:
        logs = logs.filter(created_at__date__gte=date_from)
    except Exception:
        pass
if date_to:
    try:
        logs = logs.filter(created_at__date__lte=date_to)
    except Exception:
        pass

# CLEAR ALL LOGS LOGIC (Chairperson Only)
if request.method == 'POST' and request.POST.get('action') == 'clear_logs':
    if request.session.get('admin_role') == 'chairperson':

        # Wipes all records from the database table
        ActivityLog.objects.all().delete()

        # Immediately logs the deletion itself
        admin = get_current_admin(request)
        ActivityLog.objects.create(
            id=uuid.uuid4(),
            admin=admin,
            action='LOGS_CLEARED',
            description='Admin cleared all activity logs',
        )
        messages.success(request, 'All activity logs cleared.')
        return redirect('admin_panel:activity_logs')
    else:
        messages.error(request, 'Only the chairperson can clear activity logs.')

# Fetch unique actions and admins to populate the UI dropdown filters
distinct_actions = ActivityLog.objects.values_list('action',
flat=True).distinct().order_by('action')

```

```

        action='LOGS_CLEARED',
        description='Admin cleared all activity logs',
    )
    messages.success(request, 'All activity logs cleared.')
    return redirect('admin_panel:activity_logs')
else:
    messages.error(request, 'Only the chairperson can clear activity logs.')

# Fetch unique actions and admins to populate the UI dropdown filters
distinct_actions = ActivityLog.objects.values_list('action',
flat=True).distinct().order_by('action')
all_admins = Admin.objects.filter(is_active=True).order_by('name')

# Pagination handling
paginator = Paginator(logs, 25)
page_obj = paginator.get_page(request.GET.get('page'))

return render(request, 'admin_panel/activity_logs.html', {
    'logs': page_obj,
    'page_obj': page_obj,
    'action_filter': action_filter,
    'admin_filter': admin_filter,
    'date_from': date_from,
    'date_to': date_to,
    'distinct_actions': distinct_actions,
    'all_admins': all_admins,
    'admin_role': request.session.get('admin_role', 'vits'),
})

```

ADMIN VITS OFFICER CODE SNIPPET

1. Event Scanning (Check-In & Check-Out) (vits/rep)

```

@admin_required
@require_POST
def scan_qr_api(request, event_id):
    event = get_object_or_404(Event, id=event_id)

    # ... validation for pending/closed events ...

    # --- CHECK-OUT MODE (event status is 'ended') ---
    if event.status == 'ended':
        existing = AttendanceLog.objects.filter(student=student, event=event).first()
        if not existing:
            return JsonResponse({'success': False, 'message': f'{student.name} was
not checked in.'})

        if existing.scanned_out_at:
            return JsonResponse({
                'success': False,
                'message': f'{student.name} already checked out at
{existing.scanned_out_at.strftime("%I:%M:%S %p")}.',
            })

        # Records the scan-out time
        existing.scanned_out_at = dj_timezone.now()
        existing.save(update_fields=['scanned_out_at'])
        return JsonResponse({'success': True, 'checkout': True, 'student_name':
student.name})

    # --- CHECK-IN MODE (event status is 'active') ---
    # Calculate if the student is 'present' or 'late' based on the cutoff

```

```

student.name}})

# --- CHECK-IN MODE (event status is 'active') ---
# Calculate if the student is 'present' or 'late' based on the cutoff
if event.start_time:
    event_start = dj_timezone.make_aware(datetime.combine(event.date,
event.start_time))
    cutoff      = event_start + timedelta(minutes=event.late_cutoff_mins)
    status      = 'present' if dj_timezone.now() <= cutoff else 'late'
else:
    status = 'present'

existing = AttendanceLog.objects.filter(student=student, event=event).first()
if existing:
    # Update pre-marked absent to present/late
    existing.status      = status
    existing.scanned_at = dj_timezone.now()
    existing.save(update_fields=['status', 'scanned_at'])
else:
    # Create new log
    AttendanceLog.objects.create(
        student=student, event=event, status=status, scanned_at=dj_timezone.now()
    )
return JsonResponse({'success': True, 'status': status, 'student_name':
student.name})

```

2. Required password change (vits/rep)

Step A: The Security Check (Decorator) This prevents an admin from visiting any other page until they change their password

```

def admin_required(view_func):
    @wraps(view_func)
    def wrapper(request, *args, **kwargs):
        if not request.session.get('admin_id'):
            return redirect('admin_panel:login')

        # This line blocks access if force_password_change is True
        if request.session.get('force_password_change'):
            return redirect('admin_panel:force_password_change')

        return view_func(request, *args, **kwargs)
    return wrapper

```

Step B: The Password Update Logic This snippet validates the new password (length and special characters) and updates the database.

```
def force_password_change_view(request):
    if not request.session.get('admin_id'):
        return redirect('admin_panel:login')

    if request.method == 'POST':
        new_pw = request.POST.get('new_password', '').strip()
        confirm = request.POST.get('confirm_password', '').strip()

        # Validation logic
        if len(new_pw) < 8:
            messages.error(request, 'Password must be at least 8 characters.')
        elif not re.search(r'[@#$%^&*(),.?":{}|<>]', new_pw):
            messages.error(request, 'Password must contain at least one special character.')
        elif new_pw != confirm:
            messages.error(request, 'Passwords do not match.')
        else:
            admin = get_current_admin(request)
            if admin:
                # Update password and set force_password_change to False
                admin.password_hash = hash_password(new_pw)
                admin.force_password_change = False
                admin.save(update_fields=['password_hash', 'force_password_change'])

            # Update session so they are no longer blocked
```

REPRESENTATIVE SNIPPET

1. How a Representative is assigned to a Section

When the Chairperson creates a new officer account, they select specific sections. The system then links that admin to those sections using the AdminSection model.

```

# From admins_manage_view (Creation Logic)
if action == 'add':
    # ... basic admin creation ...

    # Check if the role is 'representative'
    if role == Admin.Role.REPRESENTATIVE:
        section_ids = request.POST.getlist('section_ids') # Get selected sections
from the form
        for sid in section_ids:
            try:
                section = Section.objects.get(id=sid)
                # Create the assignment record
                AdminSection.objects.get_or_create(
                    id=uuid.uuid4(),
                    admin=new_admin,
                    section=section,
                )
            except Section.DoesNotExist:
                pass

```

2. How their view is restricted (Data Privacy)

The system uses the `admin_role` and `assigned_admins` relationship to filter every page. If the user is a Representative, the code limits the database query to only show their assigned sections.

A. Dashboard

B.In the Students List:

```
# From dashboard_view
current_admin = get_current_admin(request)
admin_role = request.session.get('admin_role', 'vits')

# Default: See all sections
assigned_sections = Section.objects.all()

# Restriction: If role is representative, filter the list
if admin_role == 'representative':
    assigned_sections = Section.objects.filter(assigned_admins__admin=current_admin)

# Only students from these specific sections are counted
total_students = Student.objects.filter(status='active',
section__in=assigned_sections).count()
```

By using `.filter(section__in=assigned_sections)`, the system guarantees that a Representative cannot see the data, student profiles, or attendance records of any section they were not explicitly assigned to by the Chairperson.

STUDENT SNIPPET

1. Sign Up: Data Storage & Duplicate Prevention

```

# From register_view
if form.is_valid():
    sid = form.cleaned_data['student_id']

    # PREVENT DUPLICATES: Checks if ID is already in the database
    if Student.objects.filter(student_id=sid).exists():
        messages.error(request, 'A student with that ID already exists.')
    else:
        # DATA STORAGE: Creates the new student record with status 'pending'
        Student.objects.create(
            id=uuid.uuid4(),
            first_name=form.cleaned_data['first_name'],
            last_name=form.cleaned_data['last_name'],
            student_id=sid,
            section=form.cleaned_data['section'],
            email=form.cleaned_data['email'],
            password_hash=hash_password(form.cleaned_data['password']),
            id_photo_path=id_photo_path,
            status='pending', # Requires Admin approval
        )

```

2. Required ID Upload

```

# From register_view
if 'id_photo' in request.FILES:
    photo = request.FILES['id_photo']

    # MAGIC BYTE VALIDATION: Checks if the file is a real image
    is_valid, error_msg = validate_upload(photo)
    if not is_valid:
        messages.error(request, error_msg)
        return render(request, 'student_portal/register.html', {...})

    # SAVING FILE: Stores the image and saves the path to the DB
    filename = f'id_photos/id_{sid.replace("-", "_")}{ext}'
    saved_path = default_storage.save(filename, photo)
    id_photo_path = f'/media/{saved_path}'

```

3. Password Creation, Change & Profile Update

The system uses **PBKDF2 Hashing** for passwords and enforces security rules (minimum 8 characters + special characters).

```
# From profile_view (Update Password/Profile)
if action == 'upload_photo' or 'profile_picture' in request.FILES:
    # Logic to update profile picture...
    student.id_photo_path = default_storage.url(saved_path)
    student.save(update_fields=['id_photo_path'])

# CHANGE PASSWORD: Checks security requirements
new_pw = request.POST.get('new_password', '').strip()
if new_pw:
    if len(new_pw) < 8:
        messages.error(request, 'Password must be at least 8 characters.')
    elif not re.search(r'[@#$%^&*(),.?":{}|<>]', new_pw):
        messages.error(request, 'Password must contain at least one special
character.')
    else:
        # Hashing the new password before saving
        student.password_hash = hash_password(new_pw)
        student.save(update_fields=['password_hash'])
```

4. Login: Data Validation (Right vs. Wrong)

```

# From login_view
try:
    student = Student.objects.get(student_id=student_id_val)

    # VERIFY PASSWORD: Compares raw input with stored hash
    if not verify_password(password, student.password_hash):
        raise Student.DoesNotExist

    # STATUS CHECK: Only approved students can log in
    if student.status == 'pending':
        messages.warning(request, 'Your account is still pending approval.')
    elif student.status == 'rejected':
        messages.error(request, 'Your registration was rejected.')
    else:
        # Successful Login
        request.session['student_id'] = str(student.id)
except Student.DoesNotExist:
    messages.error(request, 'Invalid Student ID or password.')

```

5. Download QR Code

```

# From download_qr_view
def download_qr_view(request):
    qr_token = QRToken.objects.get(student=student)
    filename = os.path.basename(qr_token.qr_path)

    # Opening the file from server storage
    f = default_storage.open(filename, 'rb')
    image_data = f.read()
    f.close()

    # Sending the response as a downloadable PNG
    response = HttpResponse(image_data, content_type='image/png')
    response['Content-Disposition'] = f'attachment;
filename="QR_Code_{student.student_id}.png"'
    return response

```

6. Dashboard Stats: Tracking Present, Late, Absent

```
# From dashboard_view
logs = AttendanceLog.objects.filter(student=student)

# Counting results based on status
total_present = logs.filter(status='present').count()
total_late     = logs.filter(status='late').count()
total_absent   = logs.filter(status='absent').count()
```

7. Upcoming Events & Section Notifications

```
# From dashboard_view
today = dj_timezone.now().date()

# LOGIC: Find active events that match the student's section
events = Event.objects.filter(
    date__gte=today,
    status='active'
).filter(
    # Only show if section matches OR if no section is specified (all sections)
    Q(expected_sections=student.section) | Q(expected_sections__isnull=True)
).distinct().order_by('date', 'start_time')
```

8. Attendance History

```
# From attendance_view
logs = (
    AttendanceLog.objects
    .filter(student=student) # Only my logs
    .select_related('event')
    .order_by('-event__date', '-scanned_at') # Newest first
)
```

GENERAL/SECURITY OF THE SYSTEM

How the QR code can be used in any event

```
# From myapp/models.py
class QRToken(models.Model):
    id = models.UUIDField(primary_key=True, default=uuid.uuid4, editable=False)

    # 1-to-1 relationship: One student has exactly ONE QR Token forever
    student = models.OneToOneField(Student, on_delete=models.CASCADE,
db_column='student_id')

    # The actual random string encoded in the QR image
    token = models.CharField(max_length=255, unique=True)
    qr_path = models.CharField(max_length=500)
    created_at = models.DateTimeField(auto_now_add=True)
```

1. Network & Infrastructure Security

DDoS Protection, Payload Limits, Security Headers, and IP Lockout

```
# --- DDoS Middleware (config/middleware.py) ---
class DDoSProtectionMiddleware:
    def __call__(self, request):
        # Global Rate Limiting: 100 requests / 60 seconds
        if request_count >= 100:
            cache.set(f'ddos_block_{ip}', True, timeout=300)
            return HttpResponse('429 Too Many Requests', status=429)

        # Payload Size Filter: Rejects bodies > 10MB
        if int(content_length) > 10 * 1024 * 1024:
            return HttpResponse('413 Payload Too Large', status=413)

        # Static Asset Exemption
        if any(request.path.startswith(p) for p in ('/static/', '/media/')):
            return self.get_response(request)

        # Security Header Enforcement
        response = self.get_response(request)
        response['X-Frame-Options'] = 'DENY'
        response['X-Content-Type-Options'] = 'nosniff'
        response['Referrer-Policy'] = 'strict-origin'
        return response

# --- IP Lockout Policy (admin_panel/views.py) ---
```

```
# --- IP Lockout Policy (admin_panel/views.py) ---
def track_login_failure(ip):
    # Tracks failed attempts; locks IP after 5 fails for 15 mins
    current = cache.get(f'login_attempts_{ip}', 0)
    cache.set(f'login_attempts_{ip}', current + 1, timeout=900)
```

2. Authentication & Identity Security

PBKDF2 Hashing, Forced Password Change, and Session Security

```
# --- PBKDF2 Hashing (admin_panel/views.py) ---
from django.contrib.auth.hashers import check_password, make_password

def verify_password(raw, stored):
    # Standard check_password uses PBKDF2-SHA256 by default in Django
    if stored.startswith('pbkdf2_'):
        return check_password(raw, stored)

# --- Forced Password Change & Complexity (admin_panel/views.py) ---
def force_password_change_view(request):
    # Password Complexity: 8 chars + special char
    if len(new_pw) < 8 or not re.search(r'[@#$%^&*(),.?":{}|<>]', new_pw):
        messages.error(request, 'Password is too weak.')
    else:
        admin.force_password_change = False # Unlocks account
        admin.save()

# --- Session Cycling (admin_panel/views.py) ---
def login_view(request):
    # ... after successful login ...
    request.session.cycle_key() # Prevents Session Fixation
```

3. Database Security

UUID v4 Primary Keys and Parameterized Queries (ORM)

```
# --- UUID v4 Primary Keys (myapp/models.py) ---
class Student(models.Model):
    # Uses random UUID strings instead of predictable numbers (1, 2, 3)
    id = models.UUIDField(primary_key=True, default=uuid.uuid4, editable=False)

# --- Parameterized Queries (admin_panel/views.py) ---
# Using the Django ORM automatically prevents SQL Injection
admin = Admin.objects.get(email=email, is_active=True)
```

4. File & Upload Security Magic Bytes, Whitelisting, and Anti-Injection

```
# --- File Security (student_portal/file_security.py) ---
def validate_upload(uploaded_file):
    filename = uploaded_file.name

    # 1. Null-byte protection
    if '\x00' in filename: return False, 'Invalid filename'

    # 2. Extension Whitelisting
    if os.path.splitext(filename)[1].lower() not in ('.jpg', '.png', '.webp'):
        return False, 'File type not allowed.'

    # 3. Anti-Double Extension check
    if any(f'.{p}' in DANGEROUS_EXTENSIONS for p in filename.split('.')[:-1]):
        return False, 'Suspicious file name.'

    # 4. Magic Byte Validation (Real File Content Check)
    header = uploaded_file.read(16)
    for offset, magic in MAGIC_SIGNATURES.get(ext):
        if header[offset:offset + len(magic)] != magic:
            return False, 'File content mismatch (Not a real image).'
```

5. DDoS Specific Controls Bouncer Logic and Asset Protection


```
# --- The "Bouncer" Logic (config/middleware.py) ---  
# Window: 60 seconds  
# Limit: 100 requests  
rate_key = f'ddos_rate_{ip}'  
request_count = cache.get(rate_key, 0)  
  
if request_count >= 100:  
    # 5-minute total block penalty  
    cache.set(f'ddos_block_{ip}', True, timeout=300)
```